Enhancing LLMs for Power System Simulations: A Feedback-Driven Multi-Agent Framework

Mengshuo Jia, *Member, IEEE*, Zeyu Cui, and Gabriela Hug, *Senior Member, IEEE*

***Abstract*—The integration of experimental technologies with large language models (LLMs) is transforming scientific research. It positions AI as a versatile research assistant rather than a mere problem-solving tool. In the field of power systems, however, managing simulations — one of the essential experimental technologies — remains a challenge for LLMs due to their limited domain-specific knowledge, restricted reasoning capabilities, and imprecise handling of simulation parameters. To address these limitations, this paper proposes a feedback-driven, multi-agent framework. It incorporates three proposed modules: an enhanced retrieval-augmented generation (RAG) module, an improved reasoning module, and a dynamic environmental acting module with an error-feedback mechanism. Validated on 69 diverse tasks from DALINE and MATPOWER, this framework achieves success rates of 93.13% and 96.85%, respectively. It significantly outperforms ChatGPT 4o, o1-preview, and the fine-tuned GPT4o, which all achieved a success rate lower than 30% on complex tasks. Additionally, the proposed framework also supports rapid, cost-effective task execution, completing each simulation in approximately 30 seconds at an average cost of 0.014 USD for tokens. Overall, this adaptable framework lays a foundation for developing intelligent LLM-based assistants for human researchers, facilitating power system research and beyond.**

***Index Terms*—Large language models, agents, power systems, simulation, retrieval-augmented generation, reason.**

I. INTRODUCTION

COMBINING laboratory automation technologies with large language models (LLMs) enables automated execution of scientific experiments [1]. Related advances span the fields of mathematics, chemistry, and clinical research, including mathematical algorithm evolution [2], geometry theorem proving [3], chemical experiment design and execution [1], as well as the development and validation of machine learning approaches for clinical studies [4]. These recent achievements signal a new research paradigm. That is, positioning AI as a research assistant for humans with natural language communication abilities, rather than merely a specialized problem solver as in the past. Establishing LLMs as research assistants also holds significant potential for advancing power systems research.

Power systems research heavily relies on simulations. To develop LLM-based research assistants in this field, LLMs must be equipped with the capability to conduct power system simulations. Enabling LLMs to execute simulation tasks has multiple implications: (i) At the assistant level, LLMs capable of conducting simulations would allow researchers to focus more on idea-intensive activities, such as simulation design, rather than on labor-intensive tasks like simulation implementation. (ii) At the interface level, LLMs conducting simulations might offer a natural-language interface. This interface can connect simulation tasks with other upstream/downstream power system tasks using natural language as the input/output. This is particularly helpful when the original inputs and outputs of these tasks are heterogeneous (e.g., different modalities), which are originally challenging to program cohesively using regular codes. (iii) At the coding level, LLMs executing simulations might be a step toward natural language coding in power systems. This might signify an evolution in programming, bringing it closer to a more intuitive, language-driven approach, a long-standing goal of programming development for decades.

However, LLMs inherently lack the capability to perform power system simulations. For recently developed simulation tools not included in LLM pre-training datasets, LLMs generally cannot execute these simulations accurately. Even for well-established tools included in pre-training data, simulation precision remains unsatisfactory. For instance, GPT-4 often has difficulty creating small distribution grids using OpenDSS [5] or writing code for simple (optimal) power flow problems [6], even though information about both OpenDSS and (optimal) power flow is available within GPT-4's pre-training dataset. While the underlying causes of this issue have not been widely discussed and recognized in the energy domain, the following factors have been proposed as potential explanations:

- *Frequency:* The low frequency of domain-specific power system knowledge in LLM training datasets — especially in the long tail of rarely encountered data — limits the models' ability to generalize effectively for specialized simulation tasks [7].
- *Quality:* High-quality, instruction-tuned, or query-based coding data specific to power system simulations in available open-source data is lacking. Missing explanatory

Received 21 November 2024; revised 6 March 2025 and 13 May 2025; accepted 26 June 2025. Date of publication 18 July 2025; date of current version 23 October 2025. This work was supported in part by the Research Start-Up Fund from Shanghai Jiao Tong University under Grant WH220403204; in part by the National Natural Science Foundation of China under Grant 62325306 and Grant 62273237; and in part by the Swiss National Science Foundation under Grant 221126 and Grant 225155. Paper no. TSG-02008-2024. *(Corresponding author: Mengshuo Jia.)*

Mengshuo Jia was with the Power Systems Laboratory, ETH Zürich, 8092 Zürich, Switzerland. He is now with the Department of Automation, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: m.jia@sjtu.edu.cn).

Zeyu Cui is with the Tongyi Lab., Alibaba Group, Beijing 100027, China (e-mail: cuizeyu15@gmail.com).

Gabriela Hug is with the Power Systems Laboratory, ETH Zürich, 8092 Zürich, Switzerland (e-mail: hug@eeh.ee.ethz.ch).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TSG.2025.3589114.

Digital Object Identifier 10.1109/TSG.2025.3589114

code annotations make it difficult for LLMs to fully contextualize and operationalize power system simulations.

- *Complexity:* Complex power system simulations require multi-step reasoning, which is inherently challenging. This difficulty increases when the model's learned patterns contain sparse or ambiguous representations related to power system simulations.
- *Precision:* Precise identification of simulation parameters, functions, and their logical connections, poses high demands on LLMs, especially when LLMs' knowledge about simulations is incomplete or fragmented. This may result in a semantic drift, causing LLMs' code generation to gradually deviate from the accurate version.

The challenges outlined above can be grouped into three main limitations: (i) limited simulation-specific knowledge, (ii) restricted reasoning capabilities for simulation tasks, and (iii) imprecision in function and option application. Enhancing the simulation capability of LLMs requires addressing these limitations. However, *few existing works have explicitly focused on overcoming the above barriers to improve the simulation capability of LLMs*, even though LLM applications in power systems are growing rapidly. Specifically, in power systems, LLMs have been recently used to translate language-based rules into mathematical constraints to facilitate optimal power flow (OPF) analysis, bridging the gap between rule-based and computational methods [8]. Researchers have also leveraged LLMs to interpret decision-making processes in real-time markets, enhancing the transparency of deep reinforcement learning systems by revealing decision rationales [9]. Additionally, LLMs have been employed to retrieve and summarize documents in response to specific power system queries [10]. In other work, LLMs have been applied to derive OPF solutions iteratively by utilizing historical cost-solution data [10]. LLMs have also facilitated gathering user preference w.r.t. electric vehicle charging, where user inputs are integrated to refine functions for EV charging optimization problems [10]. Furthermore, by combining LLMs with retrieval-augmented generation (RAG), researchers develop a carbon footprint accounting system capable of dynamically retrieving and integrating real-time, domain-specific carbon data [11]. Moreover, for cybersecurity applications, LLMs have played a role in anomaly detection to enhance system security [12]. In forecasting, LLMs, such as LLaMa2, have been used to integrate social event data into time series models, enhancing the accuracy and contextual relevance of predictions, for example, in electricity demand [13]. Also, LLMs have supported perception analysis by evaluating media sentiment and public acceptance levels of solar power initiatives, providing insights into public opinion trends [14]. Moreover, a comprehensive benchmarking framework for LLMs has been proposed for the energy domain in [15], helping to establish standardized evaluation criteria for LLMs in energy-related applications. On the other hand, potential cybersecurity threats, arising due to the application of LLMs in power systems, have also been analyzed and summarized [16].

Despite the various applications mentioned above, only a few studies have focused directly on using LLMs for power system simulations. These studies, however, only focus on conceptualizing the potential of LLMs in the simulation field [17], showcasing their current capabilities [10], [17], and assessing their effectiveness in generating general-purpose code for power system studies [5], [6]. While these studies offer valuable insights, they did not address the limitations pointed out earlier that limit LLMs' simulation performance. Although the standard RAG approach used in [6], [17], [18] can indeed enable LLMs to incorporate external power systems knowledge, it is unsuitable for simulation tasks. This happens because the standard RAG retrieves information based on the entire request as a single unit. As a result, it fails to capture the nuanced structure of complex simulation requests. It often conflates distinct function-related and option-related elements, leading to inefficiencies and reduced retrieval accuracy. Consequently, existing works fall short of systematically developing and advancing LLMs' capability to handle complex power system simulations.

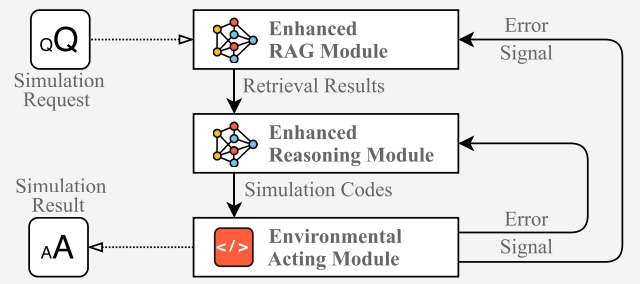

Fig. 1. The feedback-driven multi-agent framework. It consists of an enhanced RAG module, an advanced reasoning module, and an environmental acting module, all interconnected through an error-feedback mechanism. This framework enables iterative refinement by incorporating simulation-specific knowledge, improving reasoning for complex simulation tasks, and facilitating environmental interaction to generate accurate simulation results.

To bridge this gap and enhance LLMs' capability in power system simulations, this paper proposes a modular, feedback-driven, multi-agent framework. It integrates several innovative strategies, as shown in Fig. 1. Accordingly, this paper contributes in the following ways:

- It proposes an enhanced RAG module with an adaptive query planning strategy and a triple-based structure (i.e., linking options, functions, and their dependencies) for the knowledge base. This module expands the LLM's accessible knowledge in an efficient and cost-effective manner. Also, it enables LLMs to better identify and interpret simulation functions, options, and their logical relationships than the standard RAG.
- It develops an enhanced reasoning module by leveraging simulation-specific expertise, chain-of-thought prompting (CoT) and few-shot prompting. This module enables LLMs to fully understand their role, assigned tasks, reasoning pathways, and contextual knowledge (including retrieved information) in simulation tasks. Therefore, it strengthens their reasoning capabilities when generating simulation code.
- It further proposes a feedback-driven, multi-agent framework. It integrates the enhanced RAG and reasoning

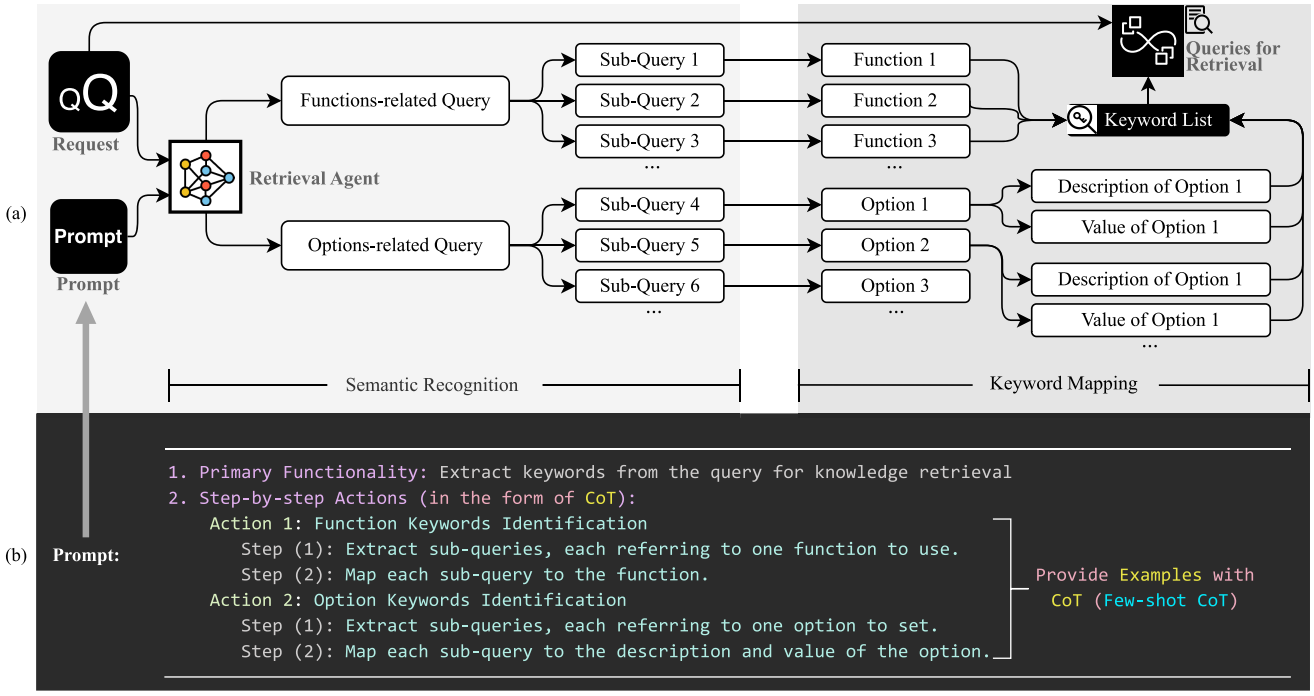

Fig. 2. Enhanced RAG module for simulation tasks. (a) The retrieval agent decomposes simulation requests into function-related and option-related sub-queries, mapped to specific functions and options for precise keyword-based retrieval. (b) Structured prompt design detailing keyword extraction steps via few-shot CoT.

modules with an environmental interaction and error-correction mechanism. This framework facilitates both action execution and feedback reception. It provides responsive error signals to initiate adaptive adjustments for the RAG and reasoning modules to automatically correct errors, thereby enhancing the reliability of simulation outcomes.

- Through testing across various strategies, simulation environments, and a diverse range of tasks, this paper reveals that even the latest LLM, o1-preview, struggles with power system simulation tasks, including those involving well-established tools like MATPOWER, despite prior exposure in the pre-training of the LLM. This paper further reveals that high simulation success rates depend on the cumulative effect of multiple strategies. Following this idea, the proposed framework demonstrates high success rates, enabling cost-effective, rapid task completion. It, therefore, provides a scalable tool for power system researchers.

This paper, as a substantial extension of the authors' preliminary work in [19], is structured as follows: Section II introduces the enhanced RAG module. Section III presents the enhanced reasoning module, and Section IV describes the environmental acting module with the feedback mechanism. Finally, Section V presents case study results, while Section VI concludes the paper with key findings and future outlook.

II. Enhanced RAG Module

As an efficient and scalable approach for integrating external knowledge to LLMs, RAG consists of three key steps: external knowledge chunking (splitting documents into smaller pieces), text embedding (converting texts into vectors using neural networks such as `text2vec`), and information retrieval (finding information in the vector space that aligns with the query) [6]. Fig. 3 illustrates a general RAG diagram. However, for power system simulations, critical questions arise: (i) *what types of queries should be used for retrieval*? and (ii) *what knowledge base should serve as the retrieval repository*? Addressing these questions reveals two primary areas for enhancing RAG's effectiveness in simulation tasks.

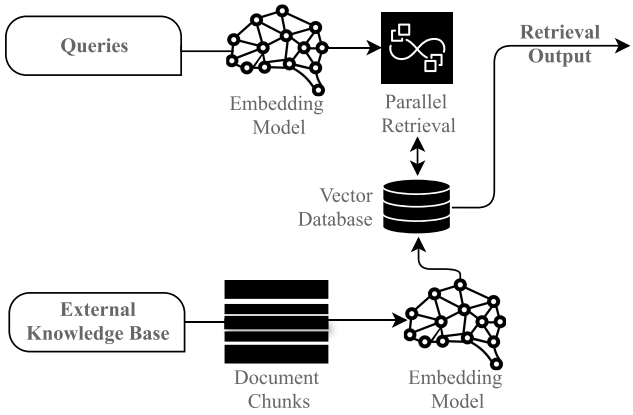

Fig. 3. General RAG diagram, including the process of external knowledge chunking, text embedding, and parallel retrieval within a vector database to produce relevant retrieval output based on the input queries and the external knowledge base. Algorithm 1 details the configuration settings and process for the embedding used in this paper.

To this end, this paper proposes an enhanced RAG module. This is specifically designed to integrate power system

simulation knowledge into LLMs and reduce hallucinations. This module emphasizes the identification of essential keywords in simulation requests to facilitate more precise knowledge retrieval than the standard RAG. It includes two main components: (i) an adaptive query planning strategy, and (ii) a triple-based structure design for the knowledge base. Together, these components provide an enhanced RAG for complex power system simulation tasks.

A. Adaptive Query Planning

This section addresses the question of what types of queries should be used for retrieval. In the standard RAG approach, the entire simulation request is processed as a single unit. As discussed and demonstrated in case studies, this conflates distinct elements in the request. As a result, it leads to inefficiencies and reduced retrieval accuracy. In fact, simulation requests typically contain two critical elements: the functions to be used and the options to be set. Thus, this paper proposes using functions and options as distinct retrieval queries. However, these elements are rarely stated explicitly in simulation requests; instead, they are embedded in natural language descriptions.

To address this, an agent-driven adaptive query planning strategy is developed, as shown in Fig. 2(a), which automatically extracts function-related and option-related queries from the broader request to serve as retrieval keywords. The strategy operates in two phases: semantic recognition and keyword mapping, carried out by a retrieval agent (e.g., a general LLM). In the semantic recognition phase, the agent categorizes the request into two query types: function-related and option-related. Each function-related query is then decomposed into sub-queries, each corresponding to a potential simulation function to be used. Similarly, option-related queries are broken down into sub-queries, each linked to a potential option to be configured. This systematic separation ensures independent processing of each component within the request.

Following semantic recognition, the keyword mapping phase aligns each identified function and option sub-query with its precise keywords. For functions, the keywords are the functions' names. For options, sub-queries are further associated with their respective descriptions and values. Ultimately, the extracted functions, option descriptions, and values are entered as parallel retrieval queries.

To enable the retrieval agent to perform both semantic recognition and keyword mapping effectively, this paper designs a structured, general action prompt that integrates chain-of-thought prompting (CoT) [20] and few-shot prompting [21] (i.e., few-shot CoT), as depicted in Fig. 2(b). Only the few-shot examples are tool-dependent, making them modular and easily adaptable. The rest of the prompt remains general and independent of specific simulation tools.

Remark 1: The adaptive query planning strategy goes well beyond simple keyword extraction. It employs semantic recognition via few-shot CoT, which can translate descriptions into relevant keywords. This enables the retrieval agent to infer implicit simulation functions and options from context, even when explicit terms are missing. For example, consider the request: "*For the IEEE 24-bus Reliability Test System, solve the AC optimal power flow (OPF) problem with the de-commitment of expensive generators*," which is the initial request in complex task 7 for MATPOWER (see Section V for more details). Rather than extracting generic phrases like "AC optimal power flow" or "de-commitment," the retrieval agent—guided by few-shot CoT prompting for function mapping—directly identifies "runuopf" as the keyword, i.e., the specific MATPOWER function for OPF with de-commitment.

Algorithm 1: Embedding for External Documents

Input:
datasetPath: Paths to external documents
chunkSize: Words per chunk
embeddingModel: Chosen text embedding model
apiKey: Credential for embedding
Output:
vectorDB: Vector database for documents

```
// Step 1: Chunk;
chunks ← splitIntoChunks(datasetPath, chunkSize)
// Step 2: Embedding;
vectors ← embedChunks(chunks, embeddingModel, apiKey)
// Step 3: Store in Vector DB;
vectorDB ← storeVectors(vectors, apiKey)
```

B. Triple-Based Structure Design for Knowledge Base

This section addresses the question of which knowledge base should serve as the retrieval repository. While each power system simulation tool includes a user manual with detailed instructions, this manual is not an ideal retrieval repository. The reasons are twofold: (i) User manuals are designed for human readability rather than automated retrieval; although readable, they are unstructured and inefficient for machine-driven queries, especially when manuals primarily consist of formulas, tables, and figures. (ii) The main challenge for LLMs in generating simulation code is understanding the logical dependencies between options and functions, as many options are function-dependent. Using only the user manual for retrieval fails to capture these complex relationships effectively.

To overcome these issues, this paper proposes an additional, easy-to-construct retrieval repository: a triple-based structured option document. The following details the approach of building such a document for a given simulation tool:

- *Step 1:* ChatGPT-4o is employed to parse the simulation manual and automatically extract key option entities. This process produces a list of options directly from the manual, including option names, potential default values/formats, and descriptions.
- *Step 2:* The same LLM is further utilized to analyze the textual context in which the options appear, thereby determining the logical relationships between

each extracted option and its associated simulation function.

- *Step 3:* The outputs from Steps 1 and 2 are organized into a structured text document using a triple format, where each triple comprises: (i) the option name with its default value/format, (ii) the corresponding function dependency (linking each option and its related functions), and (iii) the description of the option along with the choices of its value.
- *Step 4:* Finally, domain experts review and refine the automatically generated document to ensure that the logical associations between options and functions are accurate.

Note that the inclusion of the function dependency in the document enables retrieval for logical relationships. As will be demonstrated in case studies, this supplementary repository significantly enhances retrieval efficiency and improves the accuracy of simulation code generation by preserving the logical context. For detailed templates, please refer to the examples of retrieval documents provided in https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip.

C. Adaptability

The enhanced RAG module demonstrates high adaptability to a wide range of power system simulation tools. This flexibility is achieved by capturing the core principles of simulation coding—the use of functions and options. Specifically, the module decomposes simulation requests into distinct queries related to functions and options. This decomposition is tool-independent, thereby enabling the seamless integration of new simulation platforms. Additionally, the construction of a triple-based option document—linking options to their corresponding functions and dependencies—leverages inherent logical relationships that are common across different simulation tools. Together, these design choices underscore the module's robust adaptability in diverse simulation environments.

III. Enhanced Reasoning Module

The enhanced RAG module provides LLMs with retrieval results tailored to a simulation request. However, it is still essential to strengthen the LLM's reasoning abilities. This ensures the generation of correct simulation codes based on the retrieval results. Hence, a coding agent is required (i.e., another LLM) to write code for simulation tasks. This agent needs to fully understand its role, assigned tasks, reasoning path, and contextual knowledge, including retrieval results, when handling simulation tasks.

To address this, this paper proposes an enhanced reasoning module, as detailed in Fig. 4. It provides structured guidance, sequential reasoning steps, and contextual knowledge to support accurate code generation by the coding agent. Details are as follows.

A. Role and Functionality Definition

The agent deployed in this module is designated as a simulation coding agent for a specific simulation tool. Its primary function is to generate syntax-compliant simulation code that aligns with the specific task requirements, the static provided knowledge, and the dynamically retrieved knowledge.

B. Reasoning Framework

To enable systematic, tool-independent reasoning, this paper develops a few-shot CoT framework, which breaks down the simulation task into the following universal actions:

- *Function Identification:* Determines the functions relevant to the simulation task.
- *Function Syntax Learning:* Acquires the correct syntax for identified functions to ensure compliance with the simulation tool's requirements.
- *Option Information Extraction:* Identifies options and extracts their formats, values, and dependencies to maintain coherence with the selected functions.
- *Code Generation:* Integrates all extracted information into cohesive simulation code that meets task specifications and adheres to syntax requirements.

Each of these actions is further clarified with tool-specific coding examples in the prompt. While the examples are tool-dependent, the rest of the framework remains general. Overall, the above reasoning framework highlights again that the key to handling simulation tasks is: correctly identifying and combining functions and options.

C. Knowledge Integration

The above reasoning actions heavily rely on information drawn from both the simulation request and supplementary knowledge, comprising:

- *Static Basic Knowledge:* Supplies the agent with foundational information on essential functions and syntax rules pertinent to the simulation tool. This static knowledge serves as a base reference and reminder for the agent to consult when generating code. Note that such knowledge is tool-dependent.
- *Dynamic Retrieval Knowledge:* Complements static knowledge by incorporating real-time, request-specific details. This is achieved through the integration of the aforementioned RAG module, which retrieves relevant information on option formats, values, and function dependencies for accurate code generation.

The integration of the RAG module into the LLM-based coding agent (from the reasoning module) is established through placeholders in the prompt. These placeholders serve as symbolic flags that are dynamically replaced by retrieval results before the prompt is sent to the coding agent. As shown in the "Retrieval Knowledge" section of Fig. 4(b), during runtime, the enhanced RAG module retrieves simulation-specific information and substitutes these placeholders with up-to-date details on simulation functions, options, and parameters. Consequently, the coding agent generates simulation code based on both this dynamically retrieved information and the static knowledge embedded in the prompt.

Eventually, by integrating both static and dynamic knowledge, as well as the above-structured reasoning framework, the coding agent is expected to generate accurate code to address the simulation request.

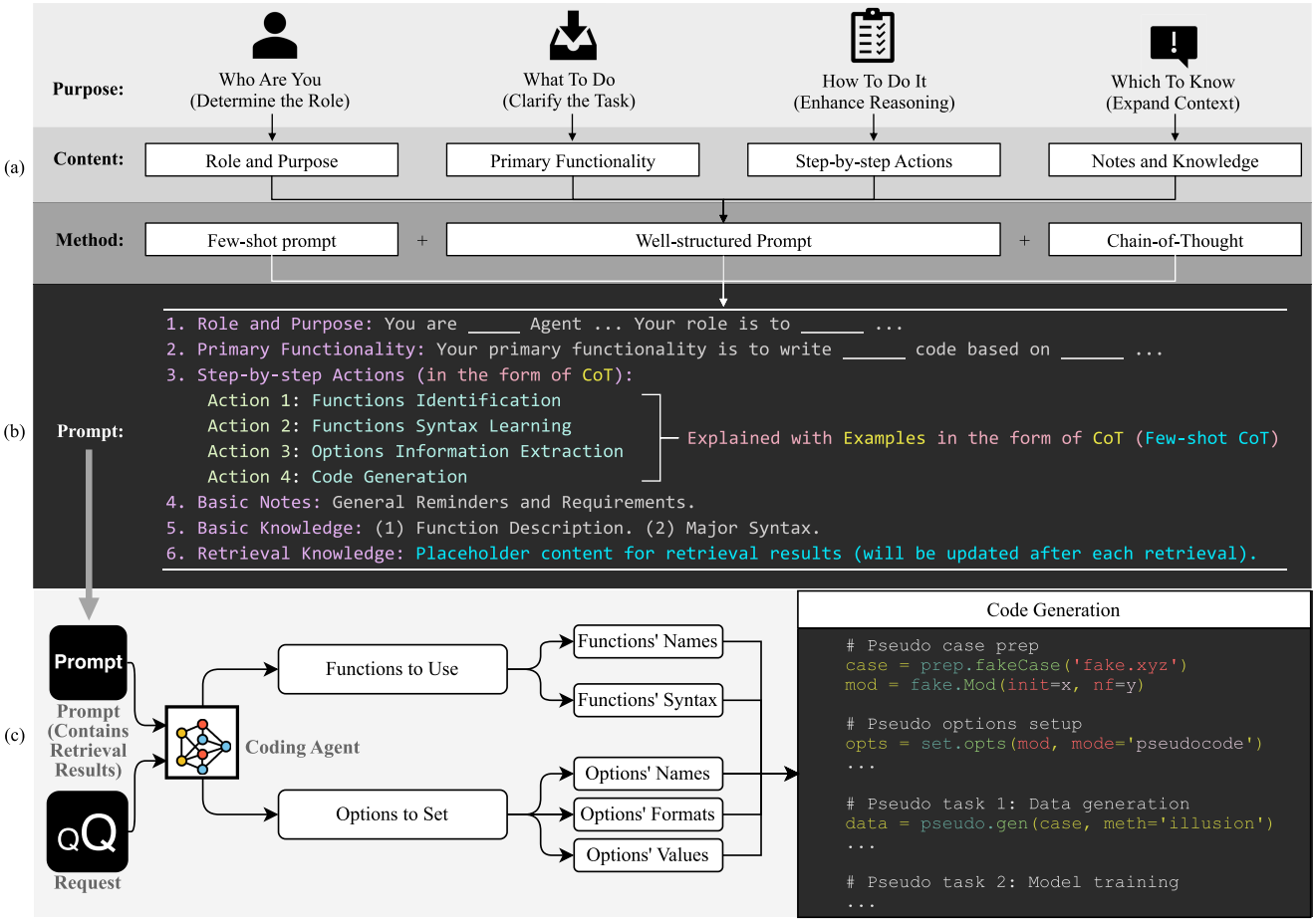

Fig. 4. Enhanced reasoning module for simulation code generation: (a) Core Concept, defining the coding agent's role, assigned tasks, reasoning path, and contextual knowledge. (b) Structured prompt design equipped with few-shot CoT, in order to enhance the agent's reasoning ability for simulation. (c) Coding agent workflow integrating the designed prompt and simulation request to produce simulation code.

D. Adaptability

The enhanced reasoning module leverages few-shot chain-of-thought prompting techniques, which embody universal principles and remain tool-agnostic. Additionally, supported by the RAG module, the reasoning module integrates two levels of knowledge: static and dynamic knowledge. The static basic knowledge provides fundamental, coarse-grained insights, such as the basic principles of using a simulation tool. Meanwhile, the dynamically-retrieved knowledge supplies detailed, fine-grained information. This dual-layer approach is universally applicable across different tools. As a result, the enhanced reasoning module adapts seamlessly to a variety of simulation tasks and environments.

IV. Environmental Acting Module With Feedback

Despite the reinforcement brought by the enhanced RAG and reasoning modules, the coding agent may still encounter errors during simulation code generation. To address this, it is essential to enable direct interaction between the LLM and the simulation environment. It allows the agent to receive execution feedback and iteratively refine its code. To this end, this paper proposes an environmental acting module with an error-feedback mechanism that integrates with both the RAG and reasoning modules, as illustrated in Fig. 5. The components of this module are described below.

A. Code Execution and Detection

The simulation code, generated by the coding agent from the enhanced reasoning module, is executed using the simulation environment API connected to a specific power system simulations tool. Following execution, the simulation environment produces results, which are then checked for error signals. Specifically:

- If an error is detected, the code advances to a stopping criterion check. If the stopping criterion is met, the process is terminated; if not, the module triggers a feedback loop with detailed error reporting.
- If no errors are detected, the process completes.

B. Error Handling and Feedback Loop

Upon detecting an error in the simulation results, an error report is automatically generated, containing:

- *Problematic Code:* The code segment that caused the error.

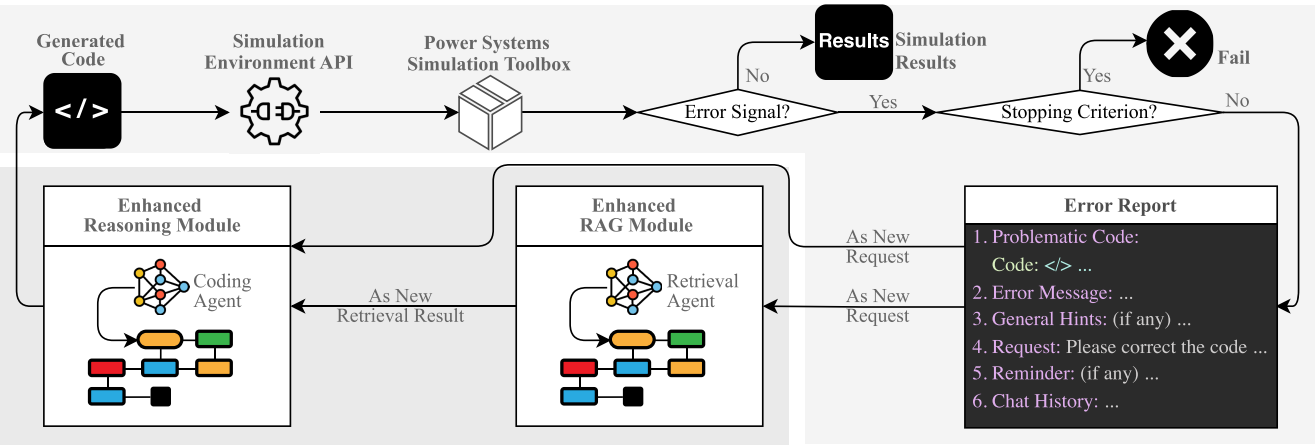

Fig. 5. Environmental acting module with an error-feedback mechanism that integrates with both the RAG and reasoning modules.

- *Error Message:* A detailed description of the error.
- *General Hints:* Additional guidance on common issues.
- *Request:* Specific corrections needed to address the error.
- *Reminders:* Additional constraints or requirements, if any.
- *Chat History:* A log of previous interactions and iterations.

C. Enhanced RAG and Reasoning Module Interplay

The error report and feedback are then processed as a new request by the retrieval agent in the enhanced RAG module. This agent retrieves relevant information based on the error report (the query planning can also be applied to error reporting by simply replacing the identification of function/option keywords with the identification of error-related keywords). The retrieved information is then passed to the enhanced reasoning module. The coding agent there uses both the retrieval results and the correction request to revise the simulation code. The updated code is fed back into the environmental acting module. This loop continues until the code meets all requirements or the stopping criterion is reached.

This design achieves two main objectives. First, the system continuously monitors simulation executions and triggers adaptive adjustments as needed, ensuring that all module interdependencies are effectively managed. Second, it handles failure modes. The error-feedback mechanism detects failures in both the RAG and reasoning modules—failures that can lead to simulation errors. Once a failure is detected, the mechanism initiates iterative corrective actions using adaptive prompts, which trigger updated retrievals and adjustments, thereby dynamically managing module failures. This conclusion is further reinforced by the case studies in Section V.

D. Adaptability

The environmental acting module features a tool-independent design. First, it leverages the error detection and reporting systems common to many power system simulation tools. Second, it formulates the error-feedback loop in a general manner, as detailed in Section IV-B. Each component is independent of any specific simulation tool. This design not only supports automatic error correction but also improves adaptability to diverse simulation platforms.

E. Pseudocode for the Full Framework

Algorithm 2 presents the complete pseudocode for the proposed framework. The algorithm employs self-explanatory pseudo-functions and pseudo-options to illustrate the core modules, including their internal processes and the interplay among them.

Remark 2: It is important to emphasize that the proposed framework is built on the premise that simulation tools are learnable by humans through sufficient documentation. If a human can use a simulation tool based on its manuals, then the proposed integrated RAG, reasoning, and feedback modules can replicate and enhance this process to improve efficiency. Hence, for scenarios where documentation is absent, it is suggested to generate necessary documents, possibly with LLMs' help. These documents can then be incorporated into the proposed framework.

V. Case Study

To comprehensively validate the proposed framework, a range of tests have been carried out, differing in three key dimensions: (i) distinct combinations of the proposed strategies within the framework to evaluate each strategy's independent effectiveness; (ii) different simulation environments, specifically DALINE [22] and MATPOWER [23], which include tools both familiar and unfamiliar to LLMs,[1] to demonstrate the framework's versatility across various applications; and (iii) a wide array of simulation tasks, spanning normal to complex scenarios, to assess the framework's performance across various simulation demands.

The following sections detail the case study configurations, followed by an analysis of the simulation outcomes for DALINE and MATPOWER. Eventually, the cost of using LLMs to perform power system simulations is discussed. All supporting materials, including

[1] DALINE is available in https://www.shuo.science/daline with a user manual in [24]. MATPOWER is available in https://matpower.org/ with a user manual in [25].

Algorithm 2: Feedback-Driven Multi-Agent Framework

Input: *modelVer*: Model version from Table II
config: Configuration settings from Table I
ragPrompt: Query planning prompt from Fig. 2
reasonPrompt: Structured reasoning prompt from Fig. 4
errTemplate: Error report template from Fig. 5
vectorDB: Vector database from Algorithm 1
task: Simulation task
maxAttempts: Max attempt number
apiKey: Credentials for LLMs
returnNum: Number of top-relevant chunks to return
Output: *simResult*: Simulation result
code: Generated code

```
// Preparation;
attemptTime ← 0;
chatHistory ← emptySet;
env ← activateSimulationEnvironment()
// Enhanced RAG Module;
retrievalLLM ← setupLLM(config, modelVer, ragPrompt, apiKey);
keywords ← generateKeywords(retrievalLLM, task);
retrievalInfo ← parallelRetrieval(retrievalLLM, keywords,
vectorDB, returnNum);
// Enhanced Reasoning Module;
reasonPrompt ← insertRetrieval(reasonPrompt, retrievalInfo);
codeLLM ← setupLLM(config, modelVer, reasonPrompt, apiKey);
[code, chatHistory] ← generateCode(codeLLM, task, chatHistory);
// Environmental Acting Module with Feedback;
[simResult, err] ← runSimulation(code, env);
while err ≠ null and attemptTime ≤ maxAttempts do
    errReport ← generateErrorReport(code, err, errTemplate);
    keywords ← generateKeywords(retrievalLLM, errReport);
    newRetrievalInfo ← parallelRetrieval(retrievalLLM,
    keywords, vectorDB, returnNum);
    reasonPrompt ← insertRetrieval(reasonPrompt,
    newRetrievalInfo);
    codeLLM ← setupLLM(config, modelVer, reasonPrompt, apiKey);
    [code, chatHistory] ← generateCode(codeLLM, errReport,
    chatHistory);
    [simResult, err] ← runSimulation(code, env);
    attemptTime ← attemptTime + 1;
if err ≠ null then
    reportFailure(code);
    saveFailedCode(code);
else
    outputResult(simResult);
    saveSimulationCode(code);
```

prompts, knowledge bases, simulation tasks, training datasets, and results (i.e., the generated codes and their benchmarks, totaling 870 coding files) are available in https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip.

A. Settings

Firstly, Table II presents the distinct combinations of the proposed strategies within the framework employed in the evaluation, with the proposed strategies shaded in gray. The model versions of LLMs can also be found in the caption of Table II, as well as in the caption of each evaluation figure/table. Meanwhile, the configurations used in all the evaluations are detailed in Table I.

Secondly, this paper selects DALINE [22] and MATPOWER [23] as the simulation environments. It is important to note that DALINE was released after the latest updates of the LLMs used in the evaluation, while the well-established tool MATPOWER was already included in the training dataset of the LLMs. Consequently, these two environments encompass both seen and unseen scenarios for the LLMs, allowing us to demonstrate the framework's versatility.

TABLE I
CONFIGURATION SETTINGS FOR EVALUATIONS (FOR THE LLM MODEL VERSIONS, PLEASE SEE TABLE I OR THE CAPTAIN OF EACH EVALUATION FIGURE/TABLE; FOR ALL THE PROMPT FILES, EXTERNAL DOCUMENTS, AND SIMULATION TASKS, PLEASE REFER TO HTTPS://GITHUB.COM/JARVISETHZ/LLM4SIMULATION/BLOB/MAIN/SUPPORTING_MATERIAL_UPDATED.ZIP)

RAG Configuration		
Parameter	**Value**	**Description**
Chunk size for .txt	30	Number of words per chunk for text files
Chunk size for .pdf	50	Number of words per chunk for PDF files
Embedding Model	`v1`	See [here] for the model `text-embedding-v1`
Return number	20	Number of top-relevant chunks to return
LLM Configuration		
Parameter	**Value**	**Description**
Temperature	0.1	Randomness (lower = more predictable)
Max Tokens	4096	Max. length of the output in tokens
Top P	1	Probability threshold for token selection
Frequency Penalty	0	Zero means common words are not suppressed
Presence Penalty	0	Zero allows natural repetition when needed
Acting Configuration		
Parameter	**Value**	**Description**
DALINE $N^t_{\max}$	3	Max. attempt number for DALINE task t
MATPOWER $N^t_{\max}$	5	Max. attempt number for MATPOWER task t

Thirdly, 34 simulation tasks were used to evaluate the proposed framework on DALINE. Similarly, for MATPOWER, 35 simulation tasks have been defined, including 8 complex tasks and 27 standard tasks.

- The simulation tasks of DALINE span from basic data generation to advanced workflows that integrate data creation, data corruption, noise handling, outlier filtering, model training, method comparison, result visualization, and full-cycle management. The simulation suite exercises all distinct functions in DALINE across various power system cases. The tasks employ multiple modeling methods (e.g., Least Squares with Huber Weighting Function, Partial Least Squares with Clustering, Ridge Regression with Voltage-angle Coupling, Locally Weighted Ridge Regression, State-independent Voltage-angle Decoupled Method with Data Correction, to name a few) and a wide array of parameters (sample sizes, base types, noise levels, outlier techniques, cross-validation settings, method hyperparameters, etc.).
- The simulation tasks of MATPOWER span standard AC and DC optimal power flow (or just load flow) analyses as well as advanced continuation power flow (CPF) studies. They employ a variety of solvers and algorithms—including Newton-Raphson, Fast-Decoupled (both XB and BX versions), Gauss-Seidel, Implicit Z-bus Gauss, MIPS, fmincon, and GUROBI—across numerous test cases such as the IEEE 9-, 14-, 30-, 57-, 118-, and

TABLE II
EVALUATED SCHEMES BY DISTINCT COMBINATIONS OF THE PROPOSED STRATEGIES; A CHECKMARK INDICATES INCLUSION OF THE CORRESPONDING STRATEGY. (GPT4O: APIS OF GPT-4O-2024-05-13 AND GPT-4O-2024-08-06, WITH THE EXACT VERSION SPECIFIED IN EACH EVALUATION; CGPT4O: CHATGPT4O; O1P: O1-PREVIEW)

	GPT4o Full	GPT4o PR	GPT4o RSR	GPT4o SR	GPT4o Sole	GPT4o NC	GPT4o NP	GPT4o NS	GPT4o NR	GPT4o NCS	GPT4o RSRNW	CGPT4o R	o1p Sole	GPT4o Sole-SFT
Query Planning	✓	✓				✓	✓	✓		✓	✓			
Triple-based Structured Option Document	✓	✓	✓	✓		✓		✓		✓	✓	✓		
Chain of Thought Prompting	✓		✓				✓	✓	✓		✓			
Few-Shot Prompting	✓		✓			✓	✓		✓		✓			
Static Basic Knowledge	✓		✓			✓	✓	✓	✓	✓	✓			
Environmental Acting and Feedback	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Proposed RAG	✓	✓				✓	✓	✓		✓	✓			
Standard RAG			✓	✓										
OpenAI's Built-in RAG												✓		
OpenAI's Built-in Supervised Fine-tuning														✓
Well-developed Error-reporting System	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓

Complex Task 1 (Daline): Generate data for 'case9' with 200 training samples and 150 testing samples. Compare and rank the accuracy of the following methods: PLS_RECW, TAY, the decoupled linearized power flow approach, RR_KPC, the ordinary least squares method, and the QR decomposition. Set the new data percentage for the method PLS_RECW to 20%, and its forgetting factor value as 0.7. Set point0 of the method TAY as 200. For the method RR_KPC, set the discrete range of tuning eta as logspace(2,5,5), and fix the random seed as 66 for RR_KPC. Set the response to {'Vm'} for all methods. Finally, use the light style for plotting the ranking, and set the type of plotting as 'probability'. Disable the plotting.	**Standard Task 16 (Daline)**: Generate data for 'case39' with 500 training samples and 250 testing samples. Train a model using LS_CLS with 5 cross-validation folds and fix the cross-validation partition. **Standard Task 21 (Daline)**: Generate data for 'case39' with 500 training samples and 250 testing samples. Visualize the linearization results for Ridge Regression with the 'academic' theme and disable the plotting.
Complex Task 8 (MATPOWER): Using the IEEE 57-bus system, perform an AC optimal power flow (OPF) analysis. Use the MATPOWER Interior Point Solver (MIPS) as the solver. Set the constraint violation tolerance to 1e-6. Set the start strategy for optimal power flow to MATPOWER decides based on solver. Ensure that voltages are represented in cartesian coordinates. Set the branch flow limits to be based on apparent power flow (MVA). Enforce the generator voltage setpoints. Additionally, ignore angle difference limits for branches. Include implicit soft limits with default values. Print no progress information during the calculation process. Set gradient tolerance for MIPS optimization to 1e-5. Set the optimality tolerance for MIPS to 1e-5. Enable step-size control for MIPS. Set the maximum number of step-size reductions per iteration of MIPS to 10. In addition, use the 51-bus radial distribution system from Gampa and Das to perform an AC power flow analysis. Use the Newton-Raphson method as the power flow algorithm. Set the mismatch tolerance for active and reactive power to 1e-8 for precise convergence.	**Standard Task 13 (MATPOWER)**: Using the 9-bus example case from Chow, perform an AC power flow analysis using the Fast-Decoupled (XB version) method. Set the maximum number of iterations to 30. Set the mismatch tolerance to 1e-8. **Standard Task 20 (MATPOWER)**: Using the 6-bus example case from Wood & Wollenberg to perform a continuation power flow (CPF) analysis. Set the original system as the load base case. Set the target case by increasing the active power demand at all buses by 50%. Enable adaptive step size control and enforce generator reactive power limits.

Fig. 6. Representative examples of simulation tasks for DALINE and MATPOWER.

145-bus systems, along with specialized cases like the 4-bus, 6-bus, 39-bus New England, 51-bus radial, 60-bus Nordic, and IEEE RTS 24-bus systems. Key parameters such as maximum iterations, mismatch tolerances, coordinate representations, and branch flow constraints were systematically varied. In the CPF analyses, both natural and pseudo arc length parameterizations were explored with adaptive step sizing, explicit nose point detection, and generation/load scaling. Additional configurations addressed generator reactive power limits, voltage setpoints, alternative formulations (current versus power balance), and solver-specific settings like gradient, optimality, and termination tolerances.

Overall, these tasks comprehensively cover both basic and advanced functionalities of DALINE and MATPOWER across a broad range of power system analysis scenarios. A selection of representative tasks is given in Fig. 6, intended as an exemplary overview; for all simulation tasks, refer to https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip.

Finally, to evaluate each scheme's performance, we compute a success rate based on the points obtained across multiple simulation tasks. Let T denote the total number of simulation tasks. For each task t ($1 \le t \le T$), let $N_{\max}^{(t)}$ be the maximum number of allowed attempts for that task and let $P_{t,i}$ be the points awarded on attempt i (with $1 \le i \le N_{\max}^{(t)}$). The scoring for task t at attempt i is defined as follows:

- $P_{t,i} = 100$ points if all requirements are fully satisfied and no irrelevant settings appear in the code.
- $P_{t,i} = 50$ points if all requirements are satisfied, but the code includes irrelevant or unnecessary settings that do not affect task completion.
- $P_{t,i} = 0$ points if any requirement is not satisfied.

Subsequent attempts are made only if the previous attempt resulted in an error, and any unused attempts are assigned the same score as the last executed attempt. Thus, the total score for task t is given by:

$$S_t = \sum_{i=1}^{N_{\max}^{(t)}} P_{t,i},$$

with a maximum possible score of $100 \times N_{\max}^{(t)}$ for that task. Finally, the overall success rate across all tasks is computed

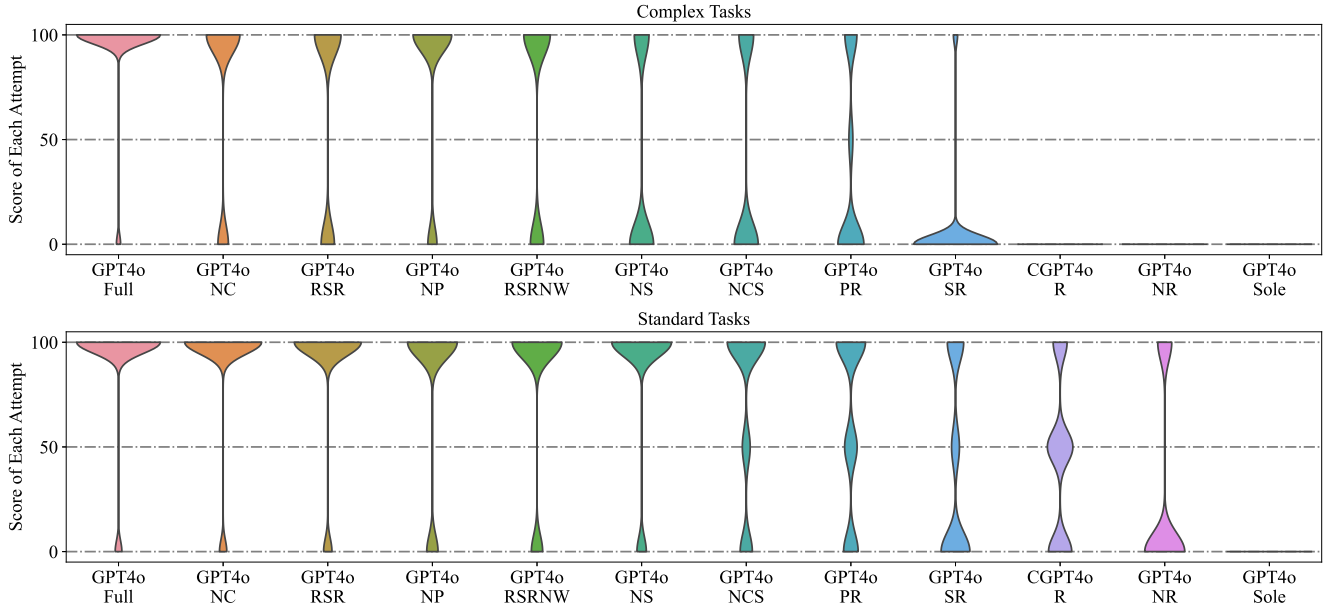

Fig. 7. Distribution of scores achieved across attempts for each evaluated scheme. For the definitions of scheme notations (e.g., `GPT4o-Full`), refer to Table II; the notations apply hereafter. (simulation environment: DALINE; GPT4o version: gpt-4o-2024-05-13; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

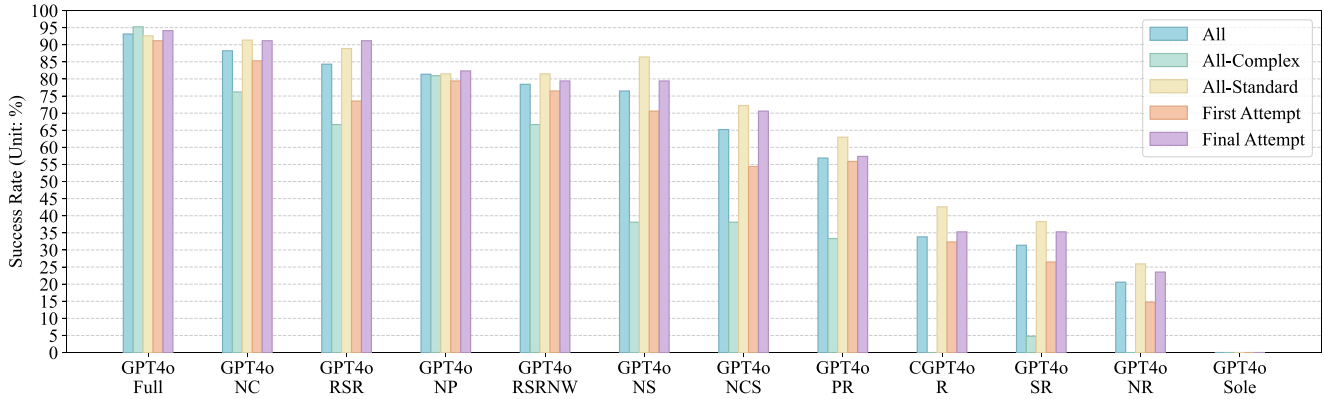

Fig. 8. Success rates for each scheme. "All" refers to the aggregated success rate across all tasks; "All-Complex" and "All-Standard" report the success rates calculated exclusively for complex and standard tasks, respectively. "First Attempt" represents the success rate achieved on the first attempt, before any error correction, and "Final Attempt" reflects the success rate after all permitted attempts are completed. The same interpretation applies hereafter (simulation environment: DALINE; GPT4o version: gpt-4o-2024-05-13; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

as:

$$R_{\text{overall}} = \frac{\sum_{t=1}^{T} S_t}{\sum_{t=1}^{T} \left(100 \times N_{\max}^{(t)}\right)} \times 100\%.$$

In the following experiments, $N_{\max}^{t} = 3$ $(\forall t)$ is set for DALINE tasks, and $N_{\max}^{t} = 5$ $(\forall t)$ is set for MATPOWER tasks, reflecting the increased complexity of the latter. In addition, the scoring was conducted manually by human experts during the evaluation process.

Remark 3: To further clarify what constitutes the aforementioned irrelevant settings, preliminary examples are provided here for illustration, with full details to be shown in the subsequent case studies. These include: (i) explicitly setting parameters to their default values when not required (e.g., in DALINE, setting `data.baseType` as `TimeSeriesRand` in Standard Task 2, and `data.program` as `acpf` in Standard Task 7, both by `GPT4o-PR`); and (ii) including unnecessary function calls, such as using `define_constants` in MATPOWER when no constants are actually used (e.g., Standard Task 27 by `GPT4o-Sole`). Such settings reflect imperfections in code generation and therefore receive partial credit.

B. Evaluation on DALINE

The evaluation results on DALINE are illustrated in Fig. 7 and Fig. 8. Fig. 7 depicts the distribution of scores achieved across attempts for each evaluated scheme, differentiating between complex and standard tasks. Fig. 8 presents the success rates for each scheme, itemized by "all tasks combined", "complex tasks only", "standard tasks only", as well as for the "first attempt success rate" and the "final attempt success rate". In the following, these evaluation results are analyzed from multiple perspectives.

1) Original Capability vs. Enhanced Capability: While equipped with environmental interaction and feedback mechanisms, `GPT4o-Sole` still demonstrates a 0% success rate for

both complex and standard tasks, indicating that GPT4o has not previously encountered DALINE. Even with a complete knowledge base supported by RAG — either through the standard RAG or OpenAI's official RAG — the resulting schemes, `GPT4o-SR` and `CGPT4o-R`, achieve success rates of only 31.37% and 33.82% across all tasks, respectively. This suggests that, even with RAG support, the latest language model, GPT4o, still lacks reliable performance in simulations. In contrast, the scheme equipped with the proposed full framework, `GPT4o-Full`, achieves a success rate of 93.13% across all tasks — a significant improvement that highlights the effectiveness of the proposed framework.

2) Fully Equipped vs. Less Equipped: The high success rate of 93.13% over all tasks for `GPT4o-Full` is due to the cumulative effects of using the complete proposed framework. Comparing other schemes with `GPT4o-Full` gives an indication of the impact of omitted strategies. For instance, although `GPT4o-NP` includes most reasoning enhancement strategies, it lacks the triple-based structured option document, resulting in a reduced success rate of 81.37%. On the other hand, omitting the few-shot CoT for reasoning, as in `GPT4o-NCS`, lowers success to 65.19%. When few-shot CoT is employed, but the proposed query planning is omitted, as in `GPT4o-RSR`, the success rate for complex tasks drops to 66.67%, particularly due to the deteriorated performance for the complex tasks. Similar comparisons can be drawn across all schemes. However, the goal is not to determine which strategy provides the highest improvement. Instead, this paper aims to emphasize that high success relies on the combined effect of multiple strategies.

3) Complex Tasks vs. Normal Tasks: In general, more complex tasks (i.e., those with multiple sub-requests) tend to increase the likelihood of errors in LLMs, resulting in lower success rates compared to standard tasks, as observed across most schemes. However, with the proposed full framework, `GPT4o-Full`, the performance gap between complex and standard tasks narrows significantly, as shown in both Fig. 7 and Fig. 8. This suggests that, with enhanced reasoning capabilities and the more effective RAG design, `GPT4o-Full` effectively identifies and addresses the sub-requests within complex tasks, similar to how it handles standard tasks. This enables LLMs to better manage complex tasks.

4) First Attempt vs. Final Attempt: The comparison between the first-attempt and final-attempt success rates demonstrates the effectiveness of environmental interaction and feedback mechanisms. As shown in Fig. 8, the final-attempt success rate is always higher than the first-attempt rate, particularly for schemes that incorporate fewer strategies from the proposed framework. These schemes typically have either reduced reasoning capability or limited retrieval information, making environmental interaction and feedback crucial for error correction. However, for `GPT4o-Full`, the difference between first-attempt and final-attempt success rates is relatively small, as `GPT4o-Full` often completes the DALINE simulation task successfully on the first attempt. This further highlights the effectiveness of the proposed framework. One noteworthy point is that the effectiveness of automatic error correction depends partly on the quality of the simulation tool's error-reporting system. Specifically, it matters whether the system provides clear, code-specific error messages. This feature affects the LLM's capability to interpret and resolve issues in the generated code. In the absence of such a feature, as with `GPT4o-RSRNW`, the success rate drops to 78.43%, with negligible improvement between the first and final attempts. This underscores that without a well-developed error-reporting system, iterative refinement may yield limited benefit. Although tools like DALINE and MATPOWER include well-developed error reporting (thereby achieving high correction accuracy), for other simulation tools where such systems are less developed, reinforcing them is recommended. In fact, the proposed framework also enables LLMs to detect vulnerabilities within a simulation tool's error-reporting system, particularly when mistakes occur. These errors may not stem from limitations of the LLMs or the framework itself but rather from inherent design issues within the tools.

5) Distribution of Attempts for Task Completion: To further examine the feedback mechanism, Fig. 9 presents the distribution of attempts (feedback loops) required to successfully solve complex and standard tasks. For complex tasks, baseline schemes (e.g., `GPT4o-Sole`) frequently fail or require multiple attempts, indicating that resolving complicated queries (i.e., tasks with more sub-queries and dependencies) poses significant challenges. In contrast, `GPT4o-Full` performs markedly better, with most tasks completed within one or two attempts. For standard tasks, all schemes achieve improved results, and failures are infrequent. `GPT4o-Full` again shows clear advantages, solving the majority of standard tasks in a single attempt. Even less-equipped schemes perform relatively well on standard tasks, suggesting that these tasks impose lower demands on reasoning, retrieval, and error correction, and are thus more manageable for LLMs.

Remark 4: Enhancing error-reporting systems in simulation tools is beyond this paper's scope. However, for tools with ambiguous error messages, the following is proposed:

- *If internal modifications are feasible*, a layered checking mechanism can be implemented, where an agent evaluates error reports across functional layers, identifies vulnerabilities, and iteratively refines messages.
- *If internal modifications are not feasible*, three external strategies may be employed: (i) For LLMs with long-context capabilities, sending the entire underlying code for analysis is possible but costly. (ii) A more efficient alternative is to batch extract relevant code segments—from surface-layer to deeper-layer functions—and submit them alongside error messages to the agents for analysis. (iii) An active trial-and-error mechanism based on agents can automatically test code with intentional bugs, mapping observed errors to ambiguous messages, in order to flag problematic outputs and facilitate learning.

These strategies can be seamlessly integrated into the proposed framework. Internal modifications require no changes to the framework since they occur within the simulation tool. Notably, the layered checking mechanism has already been

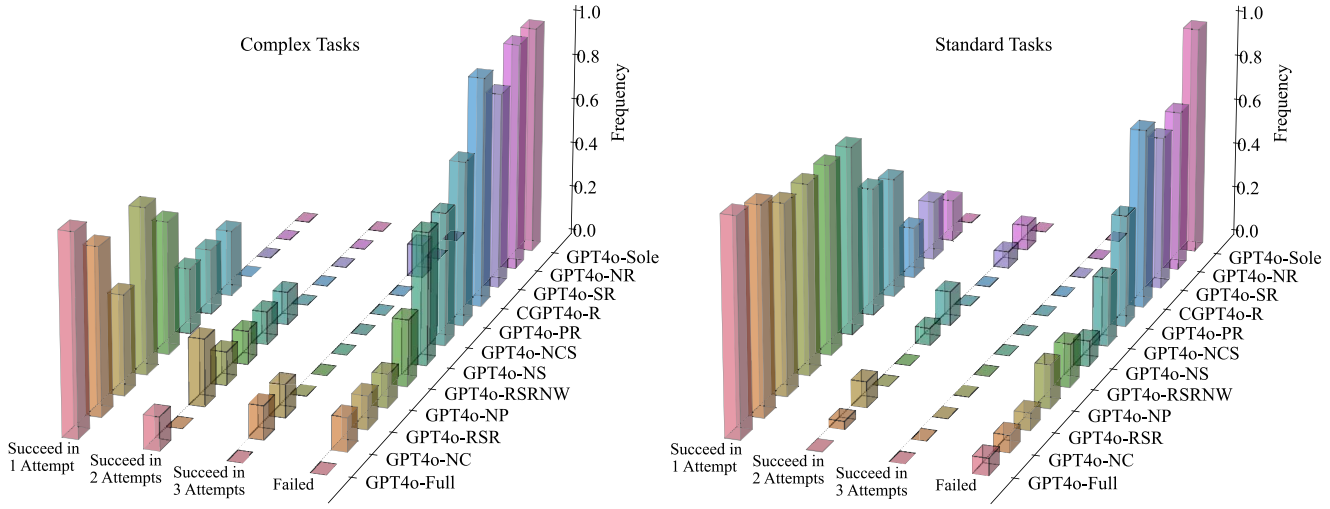

Fig. 9. Distribution of feedback loops (attempts) required for successful task completion in complex (left) and standard (right) tasks. (simulation environment: DALINE; GPT4o version: gpt-4o-2024-05-13; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

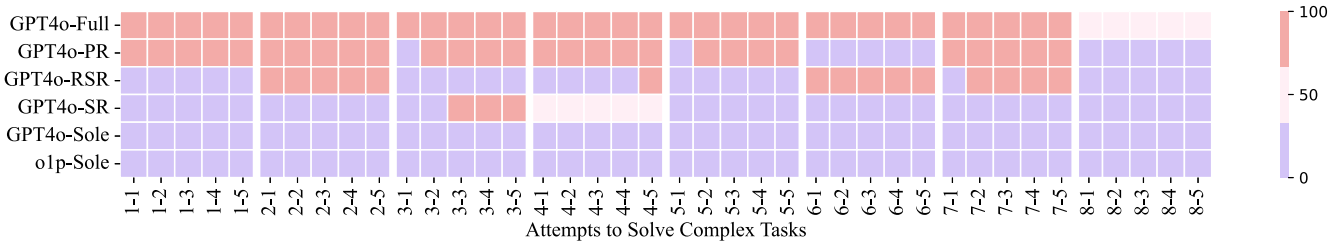

Fig. 10. Scores achieved by each evaluated scheme in individual attempts when handling complex tasks. The automatic error correction mechanism aids schemes in correcting their behavior when encountering errors in initial attempts (simulation environment: MATPOWER; GPT4o version: gpt-4o-2024-05-13; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

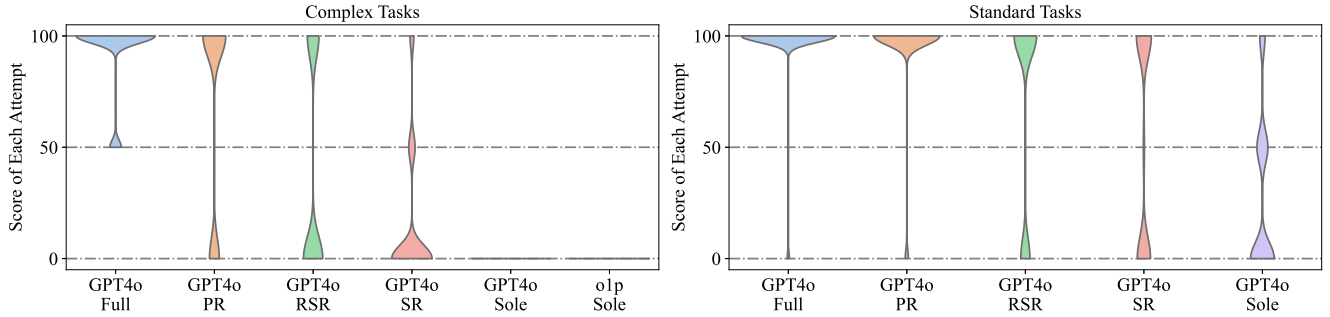

Fig. 11. Distribution of scores achieved across attempts for each evaluated scheme, separated by complex and standard tasks (simulation environment: MATPOWER; GPT4o version: gpt-4o-2024-05-13; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

implemented in DALINE, proving highly effective not only for the proposed framework but also as a robust enhancement for DALINE itself. For external strategies, the only adjustment needed is to extend the error report with relevant code extraction or error-message mapping. Future research will focus on leveraging the LLM-based trial-and-error mechanism to systematically map ambiguous error messages, reducing diagnosis overhead and improving robustness.

C. *Evaluation on* MATPOWER

The evaluation results on `MATPOWER` are illustrated in Figs. 10, 11, and 12. Specifically, Fig. 10 presents the scores achieved by each evaluated scheme in individual attempts when managing complex tasks. Fig. 11 shows the distribution of scores across attempts for each scheme, and Fig. 12 depicts the success rates of each scheme. The outcomes observed here align closely with the results on DALINE. This alignment allows us to focus primarily on comparative analyses across schemes in the subsequent discussion.

1) Original Capability vs. Enhanced Capability: Despite MATPOWER being a widely-used and well-documented tool with extensive resources available online, the latest high-performance LLMs, such as `GPT4o` and `o1-preview` (renowned for the reasoning capability), struggle to perform simulations reliably. For instance, both `GPT4o-Sole` and

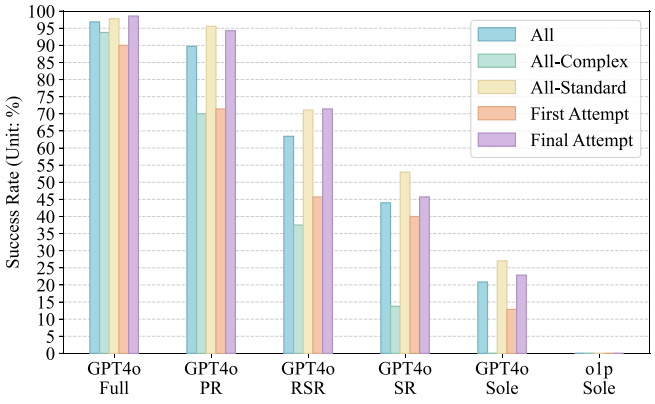

Fig. 12. Success rates for each scheme, broken down by all tasks combined, complex tasks only, standard tasks only, as well as for the first attempt success rate and the final attempt success rate (simulation environment: MATPOWER; GPT4o version: gpt-4o-2024-05-13; `o1p-Sole` is only tested by complex tasks; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

`o1p-Sole` show a 0% success rate on complex tasks, and `GPT4o-Sole` achieves only 27.77% success on standard tasks. Even with RAG and the whole knowledge base, `GPT4o-SR` reaches a success rate of only 13.75% for complex tasks and 52.96% for standard tasks. In contrast, the fully equipped framework, `GPT4o-Full`, achieves a remarkable 96.85% success rate across all tasks, with a breakdown of 93.75% on complex tasks and 97.77% on standard tasks.

2) Fully Equipped vs. Less Equipped: Consistent with the findings on DALINE, the results on MATPOWER indicate that high success rates depend on the synergistic effect of multiple strategies. For example, excluding the enhanced reasoning module, as in `GPT4o-PR`, results in an overall success rate decrease to 89.71%, with complex tasks dropping further to 70.00%. Similarly, omitting the proposed query planning strategy, as in `GPT4o-RSR`, reduces the overall success rate to 63.42% and complex tasks to 37.50%. These outcomes are substantially lower than those achieved by `GPT4o-Full`, which maintains a 93.75% success rate on complex tasks and 97.77% on standard tasks, demonstrating the critical role of each component within the proposed framework.

3) Distribution of Attempts for Task Completion: Compared to Fig. 9, a similar pattern of the attempt distribution is observed under the MATPOWER simulation environment, as shown in Fig. 13. `GPT4o-Full` still maintains a clear advantage by completing the majority of tasks in the first or second attempt. These results further confirm the effectiveness and generalizability of the proposed framework across different simulation environments.

D. Comparison With Supervised Fine-Tuning

To further verify the proposed framework's performance, it was compared with supervised fine-tuning (SFT),[2] a well-established approach known to enhance LLM performance on specific tasks—including unseen tasks such as translation and natural language inference [26]. In order to precisely demonstrate the improvements introduced by SFT, `GPT4o-Sole` has been employed as the foundation for SFT training,[3] denoted by `GPT4o-Sole-SFT` in Table II.

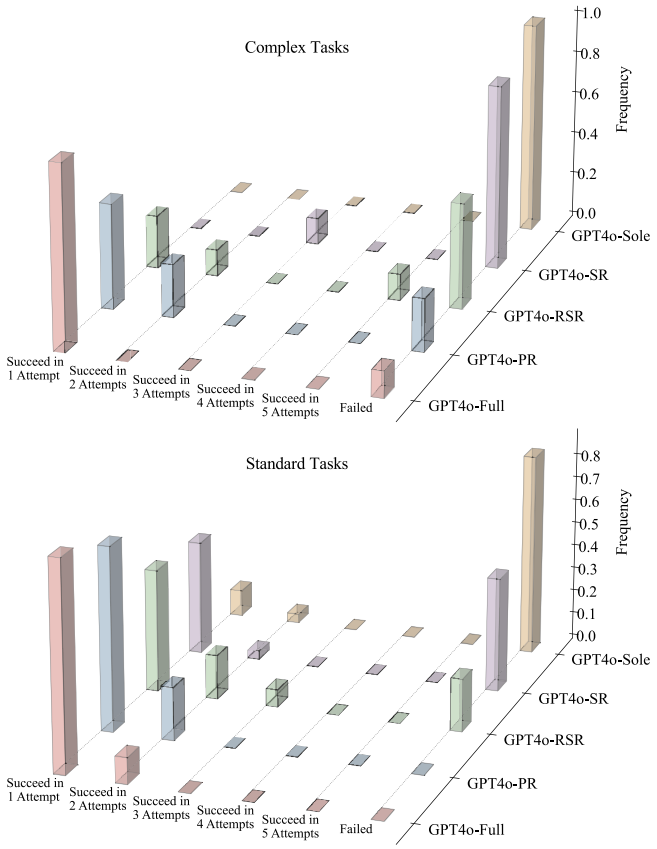

Fig. 13. Distribution of feedback loops (attempts) required for successful task completion in complex (upper) and standard (lower) tasks. (simulation environment: MATPOWER; GPT4o version: gpt-4o-2024-05-13; `o1p-Sole` is only tested by complex tasks; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

The initial comparison was conducted on MATPOWER tasks. For this evaluation, 50 random tasks (following the recommendation from OpenAI) were generated with preferred coding responses, producing a supervised training dataset of 24,249 tokens (the dataset is available from https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip). The model was fine-tuned for 3 epochs, with a batch size of 1 and a learning rate multiplier of 2 (all automatically configured). Fig. 14 illustrates the performance of `GPT4o-Full`, `GPT4o-Sole`, and `GPT4o-Sole-SFT`, based on the scores achieved in individual attempts on standard tasks. Evidently, SFT enhances the performance of `GPT4o-Sole`: `GPT4o-Sole-SFT` consistently attains more perfect scores (100 points) than `GPT4o-Sole`, with success rates of 51.11% versus 35.18%, respectively. However, `GPT4o-Full` significantly outperforms `GPT4o-Sole-SFT`, achieving 100 points for all tasks starting from the second attempt.

The underperformance of `GPT4o-Sole-SFT` could stem from the representativeness of the training datasets—despite significant human effort in generating them, these

[2]The OpenAI's SFT approach was used in this paper; see https://platform.openai.com/docs/guides/model-optimization for details.

[3]Since gpt-4o-2024-08-06 is the only version available for SFT, all GPT4o-related schemes in Section V-D use this version for consistency.

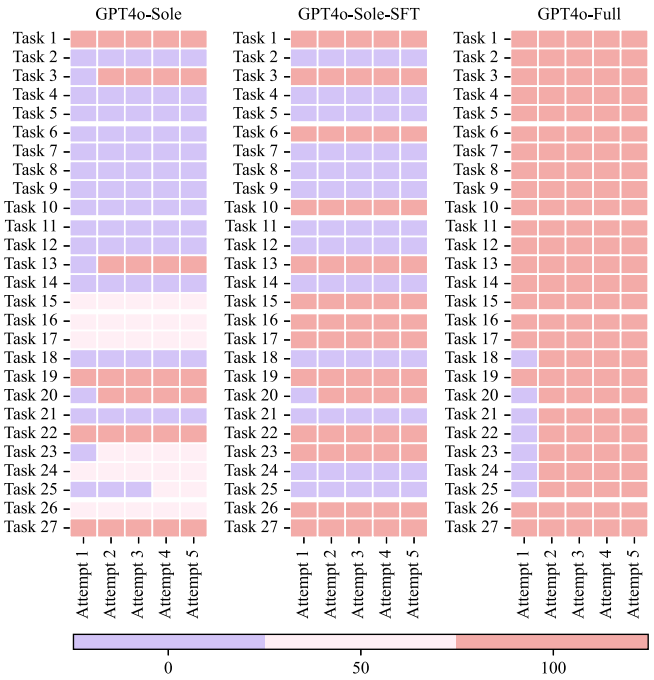

Fig. 14. Scores achieved by each evaluated scheme in individual attempts when handling standard tasks (simulation environment: MATPOWER; GPT4o version: gpt-4o-2024-08-06; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

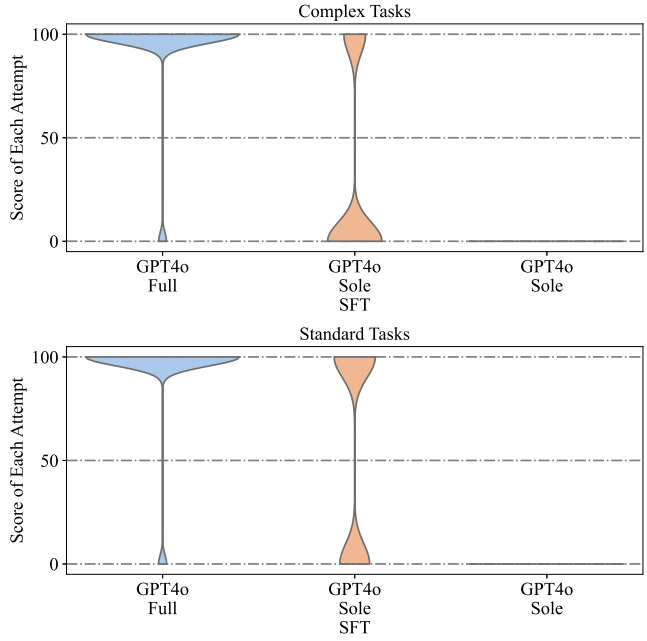

Fig. 15. Distribution of scores achieved across attempts for each evaluated scheme (simulation environment: DALINE; GPT4o version: gpt-4o-2024-08-06; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

datasets may not and cannot capture every nuanced detail of MATPOWER. To rigorously eliminate the possibility that the limited dataset is responsible for the performance gap, an additional evaluation was conducted using the DALINE simulation environment. In this experiment, the full testing dataset (i.e., the DALINE simulation tasks used for evaluation) was used as the SFT training dataset.

TABLE III
SUCCESS RATES FOR EACH SCHEME, BROKEN DOWN BY ALL TASKS COMBINED, COMPLEX TASKS ONLY, STANDARD TASKS ONLY, AS WELL AS FOR THE FIRST ATTEMPT SUCCESS RATE AND THE FINAL ATTEMPT SUCCESS RATE (SIMULATION ENVIRONMENT: DALINE; GPT4O VERSION: GPT-4O-2024-08-06; OPEN-SOURCE REPOSITORY FOR ALL TASKS AND RESULTS: HTTPS://GITHUB.COM/JARVISETHZ/LLM4SIMULATION/BLOB/MAIN/SUPPORTING_MATERIAL_UPDATED.ZIP)

Schemes	GPT4o-Sole	GPT4o-Sole-SFT	GPT4o-Full
All-Tasks	0.000%	51.961%	95.098%
All-Complex	0.000%	28.571%	95.238%
All-Standard	0.000%	58.025%	95.062%
First Attempt	0.000%	50.000%	91.176%
Final Attempt	0.000%	52.941%	97.059%

Specifically, all 34 testing simulation tasks, augmented with 16 additional tasks, were employed for fine-tuning, yielding a training dataset of 32,346 tokens (the dataset is available from https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip). `GPT4o-Sole-SFT` was fine-tuned for 3 epochs, with a batch size of 1 and a learning rate multiplier of 2 (again, all automatically configured). The score distributions and specific success rates are presented in Fig. 15 and Table III. This evaluation shows that `GPT4o-Sole-SFT` improves the success rate of `GPT4o-Sole` from 0% to 51.96% overall, with 28.57% for complex tasks and 58.03% for standard tasks. Yet, even with the entire testing dataset used for fine-tuning, `GPT4o-Sole-SFT` still fails to reach high accuracy.

The underperformance of `GPT4o-Sole-SFT` can be attributed to the inherent limitations of SFT for LLMs: SFT can capture coarse-grained information such as style, tone, format, or other qualitative aspects, but SFT cannot memorize every nuance of the training data. The reason is twofold: (1) SFT is a lossy compression process, which inevitably discards finer details, and (2) SFT only adjusts a small subset of the model's parameters, with the rest remaining focused on generating generalizable patterns — as a result, SFT cannot achieve 100% retention of task-specific details, especially for tasks like coding, which require high-precision parameters and functions. In contrast, `GPT4o-Full` leverages an enhanced RAG module, allowing the model to dynamically access and retrieve precise information from external documents. This retrieval mechanism effectively mitigates the limitations of parameter-based memorization, enabling `GPT4o-Full` to achieve over 95% accuracy across both complex and standard tasks.

E. Evaluation Under Increased Attempt Budget

To further analyze how tasks evolve toward either successful completion or termination by the stopping criterion, Fig. 16 presents the scores achieved in individual attempts with an extended attempt budget of up to 50. Meanwhile, task difficulty is explicitly characterized by the number of sub-queries within each task. This analysis offers a more comprehensive view of how different schemes perform under varying task complexities, especially when granted extensive opportunities for error correction.

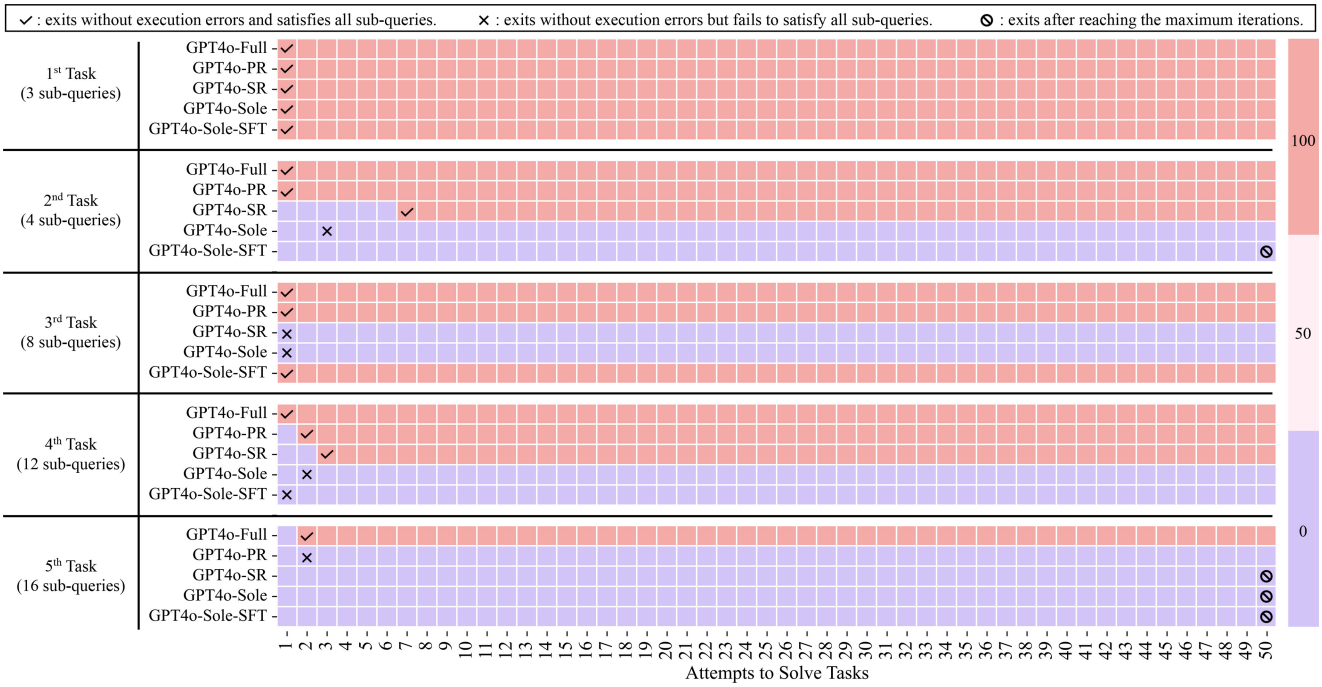

Fig. 16. Scores achieved by each evaluated scheme in individual attempts when handling tasks with different difficulty levels; the bar on the right represents the score scale, where 100, 50, and 0 indicate different score values (simulation environment: MATPOWER; GPT4o version: gpt-4o-2024-08-06; open-source repository for all tasks and results: https://github.com/JarvisETHZ/LLM4Simulation/blob/main/Supporting_Material_Updated.zip).

The results in Fig. 16 reveal a clear distinction between schemes and task difficulty levels. Baseline methods struggle significantly with complex tasks — they often either reach the maximum iteration limit (i.e., the stopping criterion) or terminate without execution errors but still fail to satisfy all sub-queries. Typical failure reasons include: (i) invalid option names, which prevent proper execution and remain unresolved even after 50 feedback loops (e.g., `GPT4o-Sole-SFT` in the 2nd and 5th tasks, `GPT4o-Sole` in the 5th task, and `GPT4o-SR` in the 5th task); (ii) correct option names with invalid parameter values produce incorrect but executable code (e.g., `GPT4o-Sole` in the 2nd to 4th tasks and `GPT4o-SR` in the 3rd task); (iii) missing critical option settings, which result in early termination without errors but produce incorrect outputs (e.g., `GPT4o-PR` in the 5th task).

These observations confirm that repeated error-feedback loops alone are insufficient for convergence on challenging tasks without strong reasoning and retrieval capabilities. In contrast, `GPT4o-Full` exhibits robust performance across all difficulty levels, completing all tasks — including those with numerous sub-queries — within only a few attempts. Notably, `GPT4o-Full` only failed at the first attempt for the 5th task, where it misused an option name. This error, however, was successfully corrected in the second attempt.

F. Cost Analysis

The cost analysis of `GPT4o-Full` for executing simulation tasks in DALINE and MATPOWER is presented in Table IV, with average values across all tasks shown. The reported time covers the entire process, including retrieval, reasoning, code generation, simulation execution, result aggregation, and, where necessary, code correction. Remarkably, `GPT4o-Full` completes each task in approximately half a minute. Additionally, the token expense per task is roughly 0.014 USD.

TABLE IV
AVERAGE COST ANALYSIS OF GPT4O-FULL PER TASK*

Environment	Time (sec.)	Input Token†	Output Token	Expense (USD) ‡
DALINE	29.446	7882.294	168.353	0.014
MATPOWER	32.703	5338.514	274.371	0.013

† A token is a fundamental unit of text, typically representing fragments of words. Tokens quantify both the input (text submitted to the LLM via API) and the output (text or code generated by the LLM via API).
‡ Expense refers to the monetary cost incurred for processing these tokens, as charged by the API provider (OpenAI in this study).
* The API provider automatically computes the total tokens and associated expenses. The average values here are calculated by dividing the total tokens (and corresponding expenses) used across all simulation tasks with `GPT4o-Full` by the number of tasks.

It is noteworthy that, aside from parallel retrieval, no specialized acceleration techniques were employed in this framework. Thus, despite its already satisfactory performance, there is considerable potential for speed enhancements. Even in the current state, without any specific acceleration strategies, `GPT4o-Full` can execute approximately 120 simulation tasks per hour in DALINE and MATPOWER, with a total token cost of around 1.68 USD. Given this high efficiency and cost-effectiveness, the proposed framework presents a promising pathway to improve researcher productivity.

It must be emphasized, however, that the reported execution time and token cost serve only as approximate indicators, given the inherent variability introduced by factors such as retrieval, reasoning, code execution, result aggregation, and iterative error correction. This analysis aims to provide readers with a general sense of the efficiency of automated simulations within the scope of the specific simulation tasks, rather than to establish precise quantitative relationships.

VI. Conclusion

This paper addresses the research gap in enhancing LLMs for power system simulations by proposing a feedback-driven, multi-agent framework. It represents the first systematic approach to significantly improve LLMs' simulation capabilities across both familiar and new tools. Validated on 69 diverse simulation tasks from DALINE and MATPOWER, the proposed framework achieved substantial performance improvements, with success rates of 93.13% and 96.85%, respectively. It far surpasses those of baseline schemes, including the latest LLM, o1-preview, and SFT for LLMs. Key findings include: (i) The original simulation capability of LLMs is limited, as evidenced by `GPT4o` and `o1-preview` achieving success rates no higher than 27.77%. (ii) Even with the standard RAG module and a comprehensive knowledge base, LLMs achieve overall success rates below 45%, highlighting the need for a more comprehensive approach. (iii) Our framework's high success rate stems from a synergistic integration of enhanced RAG, enhanced reasoning, as well as environmental acting and feedback mechanisms. Removing any of these elements results in a significant performance decline. (iv) While SFT may be able to capture coarse-grained attributes such as style, tone, and format, it struggles to retain the full detail necessary for power system simulations. Even when the entire test dataset is included in the training set, SFT achieves a success rate of less than 60%. This limitation stems from its inherently lossy compression mechanism and constrained parameter update capacity, which prevent it from encoding fine-grained details with high fidelity. (iv) The proposed framework enables LLMs to execute tasks efficiently, with each task completed in approximately 30 seconds at a token cost of only 0.014 USD (for the simulation tasks used in this paper), offering a scalable, cost-effective solution that enhances the productivity of human scientists in power systems.

Overall, the effectiveness of this work relies on the coordinated integration of general-purpose LLM capabilities (such as natural language understanding and code generation) with carefully designed domain-specific strategies tailored to power system simulations. These strategies are essential for addressing the unique challenges posed by simulation tasks. While this work focuses on simulations, the proposed framework is inherently designed to enable LLMs to adapt to unfamiliar domains and is readily extensible to broader power system research. Furthermore, the proposed simulation-enhanced LLM system may serve as a key component in future human-machine collaborative research, supporting not only simulation execution but also the validation and refinement of research ideas.

However, several critical future challenges still remain. **First**, while the proposed framework demonstrates significant improvements over existing schemes and supervised fine-tuning, achieving a perfect 100% success rate for LLM-based simulations remains an open challenge. For instance, the proposed framework sometimes fails to detect "non-execution-bug" failures, resulting in unaware, inaccurate outputs that cannot be automatically corrected. A potential avenue for future work is to explore the use of a RAG-supported, fine-tuned LLM-based code-checking agent, which would verify executed simulation code and identify hidden errors that the error-reporting mechanism may overlook. **Second**, when simulation requests are inherently ambiguous or underspecified, accurately determining the user's intent becomes difficult for any automated method, not just the framework presented in this paper. This ambiguity primarily arises from the initial lack of detail in the user's input, rather than any intrinsic limitation of natural language coding. To address this, a promising future direction is to integrate an interactive dialogue stage into the framework. In this approach, an autonomous agent would evaluate the specificity of the initial request and, if necessary, prompt the user with targeted clarifying questions. This human-in-the-loop strategy could help refine the requirements, leading to more precise retrieval and improved code generation.

Acknowledgment

The authors would like to acknowledge the assistance of ChatGPT-4o [27] for language polishing of this paper.

References

[1] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, no. 7992, pp. 570–578, 2023.

[2] B. Romera-Paredes et al., "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.

[3] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, "Solving olympiad geometry without human demonstrations," *Nature*, vol. 625, no. 7995, pp. 476–482, 2024.

[4] S. T. Arasteh et al., "Large language models streamline automated machine learning for clinical studies," *Nat. Commun.*, vol. 15, no. 1, p. 1603, 2024.

[5] R. S. Bonadia, F. C. Trindade, W. Freitas, and B. Venkatesh, "On the potential of chatgpt to generate distribution systems for load flow studies using openDSS," *IEEE Trans. Power Syst.*, vol. 38, no. 6, pp. 5965–5968, Nov. 2023.

[6] S. Majumder et al., "Exploring the capabilities and limitations of large language models in the electric energy sector," *Joule*, vol. 8, no. 6, pp. 1544–1549, 2024.

[7] H. Chang et al., "How do large language models acquire factual knowledge during pretraining?" in *Proc. 32nd Conf. Neural Inf. Process. Syst.*, 2024.

[8] Z. Yan and Y. Xu, "Real-time optimal power flow with linguistic stipulations: Integrating GPT-agent and deep reinforcement learning," *IEEE Trans. Power Syst.*, vol. 39, no. 2, pp. 4747–4750, Mar. 2024.

[9] B. Zhang, C. Li, G. Chen, and Z. Dong, "Large language model assisted optimal bidding of BESS in FCAS market: An AI-agent based approach," 2024, *arXiv:2406.00974*.

[10] C. Huang, S. Li, R. Liu, H. Wang, and Y. Chen, "Large foundation models for power systems," in *Proc. IEEE Power Energy Soc. Gener. Meeting (PESGM)*, 2024, pp. 1–5.

[11] H. Wang et al., "Carbon footprint accounting driven by large language models and retrieval-augmented generation," 2024, *arXiv:2408.09713*.
[12] A. Zaboli, S. L. Choi, T.-J. Song, and J. Hong, "Chatgpt and other large language models for cybersecurity of smart grid applications," in *Proc. IEEE Power Energy Soc. Gener. Meeting (PESGM)*, 2024, pp. 1–5.
[13] X. Wang, M. Feng, J. Qiu, J. Gu, and J. Zhao, "From news to forecast: Integrating event analysis in llm-based time series forecasting with reflection," in *Proc. 38th Conf. Neural Inf. Process. Syst.*, vol. 37, 2025, pp. 58118–58153.
[14] K. Nuortimo, J. Harkonen, and K. Breznik, "Global, regional, and local acceptance of solar power," *Renew. Sustain. Energy Rev.*, vol. 193, Apr. 2024, Art. no. 114296.
[15] X. Zhou et al., "ElecBench: A power dispatch evaluation benchmark for large language models," 2024, *arXiv:2407.05365*.
[16] J. Ruan et al., "Applying large language models to power systems: Potential security threats," *IEEE Trans. Smart Grid*, vol. 15, no. 3, pp. 3333–3336, May 2024.
[17] D. Lifu, C. Ying, X. Tannan, H. Shaowei, and S. Chen, "Exploration of generative intelligent application mode for new power systems based on large language models," *Autom. Elect. Power Syst.*, vol. 48, no. 19, pp. 1–13, 2024. [Online]. Available: https://github.com/xxh0523/llm4power
[18] P. S. H. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. 34th Conf. Neural Inf. Process. Syst.*, 2020, pp. 1–16.
[19] M. Jia, Z. Cui, and G. Hug, "Enabling large language models to perform power system simulations with previously unseen tools: A case of Daline," 2024, *arXiv:2406.17215*.
[20] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. 36th Conf. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
[21] T. Brown et al., "Language models are few-shot learners," in *Proc. 34th Conf. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
[22] M. Jia, W. Y. Chan, and G. Hug (ETH Zrich, Zurich, Switzerland). *Daline: A Data-Driven Power Flow Linearization Toolbox for Ppower Systems Research and Education*. 2024. [Online]. Available: https://doi.org/10.3929/ethz-b-000681867
[23] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. power Syst.*, vol. 26, no. 1, pp. 12–19, Feb. 2011.
[24] M. Jia, W. Y. Chan, and G. Hug (ETH Zrich, Zurich, Switzerland). *User Manual for DALINE 1.1.5*. 2024. [Online]. Available: https://doi.org/10.3929/ethz-b-000680438
[25] R. D. Zimmerman and C. E. Murillo-Sánchez (MAT POWER, Montpellier, France). *Matpower 8.0 User's Manual*. 2024. [Online]. Available: https://matpower.org/docs/MATPOWER-manual-8.0.pdf
[26] J. Wei et al., "Finetuned language models are zero-shot learners," in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–46.
[27] OpenAI. "ChatGPT-4o." Accessed: Jul. 31, 2025. [Online]. Available: https://openai.com/

Mengshuo Jia (Member, IEEE) received the B.E. degree in electrical engineering from North China Electric Power University in 2016, and the Ph.D. degree in electrical engineering from Tsinghua University in 2021. From 2021 to 2023, he was a Postdoctoral Researcher with the Department of Information Technology and Electrical Engineering, ETH Zürich, where he was exceptionally promoted to a Senior Scientist based on merit in 2023. He is currently a Tenure-Track Associate Professor with the Department of Automation, Shanghai Jiao Tong University. He also served as a Principal Investigator funded by the Swiss National Science Foundation. He is the lead developer of the open-source Daline toolbox. His research focuses on the modeling, analysis, and optimization of power systems, with particular interest in AI-powered and hybrid-driven approaches.

Zeyu Cui received the B.E. degree in electrical engineering from North China Electric Power University in 2016, and the Ph.D. degree from the Institute of Automation, Chinese Academy of Sciences in 2021. He is currently serving as an Algorithm Expert with the Qwen Team, Alibaba Group. His research focuses on large language models for coding and natural language processing, with a particular interest in agent-based coding systems.

Gabriela Hug (Senior Member, IEEE) was born in Baden, Switzerland. She received the M.Sc. degree in electrical engineering and the Ph.D. degree from the Swiss Federal Institute of Technology, Zürich, Switzerland, in 2004 and 2008, respectively. After earning her Ph.D., she worked with the Special Studies Group of Hydro One, Toronto, ON, Canada, and from 2009 to 2015, she was an Assistant Professor with Carnegie Mellon University, Pittsburgh, PA, USA. She is currently a Professor with the Power Systems Laboratory, ETH Zürich, Zürich, Switzerland. Her research is dedicated to the control and optimization of electric power systems.